

BARMA-Python: Roadmap

Everton da Costa

2026-08-17

Table of contents

Overview	1
1 Public API	1
2 Project Status & Roadmap	2
2.1 Implementation Status	2
2.2 Benchmarking	3
2.3 Next Steps	3
3 Validation & Testing	3
3.1 pytest	3

Overview

This document covers the validation and engineering status of BARMA-Python. The modeling walkthrough — data, fitting, diagnostics, and the forecast benchmark — lives in the companion report, *Modeling Relative Humidity in Brasília with the β ARMA Model* ([report_brasilia_relative_humidity](#)).

Source code: github.com/Everton-da-Costa/BARMA-Python-2026

1 Public API

The user-facing interface is organized around five actions:

- Construct: `BARMA(y, ar, ma, exog, link)`
- Fit: `.fit()`, returning a results object with `.converged` and `.n_iter`

- Inspect: `.summary()`, `.aic`, `.bic`, `.log_likelihood`, `.fitted_values`, `.fim_barma`
- Diagnose: `.residuals()`, `.plot_diagnostics()`, `.ljungbox_test()`, `.plot_ljungbox()`
- Forecast: `.forecast()`, `.plot_forecast()`

2 Project Status & Roadmap

The lists below track what is implemented and validated, and what remains.

2.1 Implementation Status

Core estimation

- Log-likelihood computation
- Score vector computation
- Starting-value estimation
- Link function structure generation
- Optimization procedure integration
- Complete β ARMA model fitting, including subset-ARMA specifications (non-contiguous AR and MA lags) and model selection via BIC
- Pure submodels: strictly AR (β AR) and strictly MA (β MA) processes
- Separation of model specification and fitted-results objects

Inference

- Fisher Information Matrix, providing standard errors, z-values, and p-values for fitted models
- AIC and BIC calculations
- Three residual types: Pearson, raw, and link-scale

Forecasting & I/O

- Forecasting and predicted-value extraction
- Data ingestion pipeline (`fetch_humidity_brasilia.py`)

Validation

- `pytest` test suite for all core functions, validating outputs against the R reference
- Worked validation against the `betaARMA` R package, with parameter estimates agreeing to machine precision at fixed parameter vectors

2.2 Benchmarking

The forecast comparison in the companion report benchmarks β ARMA against three baselines (DHR, SARIMA, and ARIMA) fitted with `auto_arima()` from the `pmdarima` package. These baselines are external references for evaluation, not part of the BARMA-Python implementation.

2.3 Next Steps

1. Package the project for distribution (PyPI), with documentation and installation instructions.
2. Extend the benchmarking section with machine-learning baselines, such as gradient-boosted trees (XGBoost/LightGBM) on lagged and calendar features, and a neural forecaster (e.g. N-BEATS via `neuralforecast`).

3 Validation & Testing

3.1 pytest

To validate the Python implementation, we designed a `pytest` suite that compares its outputs directly against those generated in R. By feeding the same time series and start values into both systems, we verified the accuracy of the log-likelihood, score vector, initial values, and the link structure generation function, using a strict tolerance of $1e-10$.

Validating the final model estimates is more challenging. Although both implementations use the BFGS algorithm, inherent differences between R's `optim` and Python's `scipy.optimize.minimize` prevent exact agreement in the final digits. For these tests, we applied a tolerance of $1e-3$, which confirms that both implementations converge to the same optimum while tolerating optimizer-level differences in the trailing digits.